

ERP System

Database Design Specification

Wholesale & Distribution ERP Platform · Version 1.0

Database: MySQL 8.0 · InnoDB · utf8mb4
Architecture: 3NF · Soft Deletes · Audit Trails · Append-Only Ledgers
Tables: 31 tables across 14 modules (no warehouse complexity)
Notation: Crow's Foot ERD · Color-Coded Modules

Prepared: June 2026 · Confidential
CONFIDENTIAL — FOR INTERNAL USE ONLY

Table of Contents

- 1. Executive Summary
- 2. System Overview
- 3. Business Workflow
- 4. Module Design
- 5. Entity Relationship Diagram
- 6. Table-by-Table Design
- 7. Customer Workflow
- 8. Inventory Workflow
- 9. Damaged Stock Workflow
- 10. Order Lifecycle
- 11. Challan & Dispatch Workflow
- 12. Returns Workflow
- 13. Finance Workflow
- 14. Notifications & Audit Strategy
- 15. Indexing & Scaling Strategy
- 16. Future Enhancements

1. Executive Summary

This document is the authoritative Version 1.0 Database Design Specification for the Wholesale & Distribution ERP System. It has been regenerated to align the written specification with the updated ER Diagram. All inconsistencies between prior documentation and the ERD have been resolved — the ERD is the source of truth.

Key Design Decisions

- No warehouse module in V1 — inventory is managed at product level only. Warehouse support is a planned V2 feature.
- No separate vehicle/driver tables — vehicle and driver details are embedded within the dispatches table for simplicity.
- Damaged stock as a first-class entity — stock_damaged table with product_id, quantity_damaged, damage_reason, recorded_by, and remarks.
- Available Stock formula: Available = on_hand – reserved – damaged. Damaged stock is never available for sale.
- Three-price model — order_items captures unit_price, dealer_landing_price, and selling_price as a snapshot at time of order.
- Remarks on all business tables — customers, vendors, products, orders, order_items, challans, challan_items, dispatches, dispatch_items, returns, return_items, payments, stock_damaged, inventory_adjustments.
- 31 tables across 14 modules — fully normalized to 3NF with FK constraints, CHECK constraints, and soft deletes.

Document Scope

Field	Value
Database Engine	MySQL 8.0, InnoDB storage engine, utf8mb4 character set
Normalization	Third Normal Form (3NF) enforced across all entities
ID Strategy	BIGINT UNSIGNED AUTO_INCREMENT on all primary keys
Soft Delete	deleted_at DATETIME NULL on all master/transactional tables
Audit Trail	audit_logs (append-only) + domain-specific history tables
Money Fields	DECIMAL(14,2) for totals; DECIMAL(12,2) for unit prices
Quantities	INT UNSIGNED (whole units); DECIMAL for fractional units
Total Tables	31 tables across 14 business modules
Warehouse	NOT included in V1 — planned for V2
Vehicles/Drivers	Inline fields in dispatches table; no separate master tables

2. System Overview

The ERP platform provides end-to-end management of a wholesale distribution business: from customer onboarding and product cataloguing, through order creation, challan generation, dispatch, and delivery, to payment collection and return processing — with full audit trails, inventory accuracy, and reorder intelligence.

Module Summary

Module	Tables	Type	V1 Scope
Identity & Access	roles, users, permissions, role_permissions, refresh_tokens	Master + Session	Full
Customer Management	customers, regions, credit_limit_history	Master + Ledger	Full
Vendor Management	vendors, vendor_contacts, vendor_product_mapping	Master	Full
Product Management	product_categories, units_of_measure, products, pricing	Master + Versioned	Full
Inventory Management	stock_on_hand, stock_reserved, stock_damaged, stock_transactions, inventory_adjustments	Snapshot + Ledger	Full (No Warehouse)

Module	Tables	Type	V1 Scope
Sales Management	orders, order_items, order_status_history	Transactional	Full
Challan Management	challans, challan_items	Transactional	Full
Dispatch Management	dispatches, dispatch_items	Transactional	Full (Inline V/D)
Returns Management	returns, return_items	Transactional	Full
Finance	payments, outstanding_balances, credit_limit_history	Financial Ledger	Full
Notifications	notifications, notification_recipients	Event-Driven	Full
Audit & Compliance	audit_logs	Immutable Ledger	Full
Reorder Intelligence	reorder_suggestions	Analytics	Basic
Documents & Imports	generated_documents, excel_import_logs	File Tracking	Basic

3. Business Workflow

Master Order-to-Cash Flow

Step	Action	Tables Involved	Stock Impact
1	Customer lookup / auto-create	customers, regions	None
2	Create Order (DRAFT)	orders, order_items	None
3	Confirm Order	orders, order_status_history	Reserve → stock_reserved +, stock_transactions RESERVED
4	Generate Challan	challans, challan_items	None
5	Create Dispatch	dispatches, dispatch_items	None
6	Mark Delivered	orders, dispatches	stock_on_hand –, stock_reserved –, stock_transactions DISPATCHED
7	Record Payment	payments, outstanding_balances	None
8	Process Return (if any)	returns, return_items	GOOD→stock_on_hand+; DAMAGED→stock_damaged+

Available Stock Formula

The system calculates available stock in real time using three independent snapshot tables: **Available Stock = stock_on_hand.quantity_on_hand – stock_reserved.quantity_reserved – stock_damaged.quantity_damaged**. Damaged stock is excluded from available stock at all times. Orders cannot be allocated against damaged inventory.

4. Module Design

Identity & Access

roles — Defines system roles (Admin, Sales Manager, Warehouse Staff). Unique code per role.

permissions — Granular permission matrix keyed by module + action + code. Used for RBAC.

role_permissions — M:N junction linking roles to permissions. Composite PK (role_id, permission_id).

users — All system actors. email unique. role_id FK to roles. Soft delete via deleted_at.

refresh_tokens — JWT refresh token store. Unique token_hash and jti. Tracks IP and revocation.

Customer Management

regions — Geographic segmentation reference table. 1 region → N customers. Unique name and code.

customers — Master buyer registry. company_name required. GST unique. Sales manager assigned. region_id FK. Credit limit tracked. Soft delete. Remarks supported.

credit_limit_history — Append-only ledger of every credit limit change: old_limit, new_limit, changed_by, reason, changed_at.

Vendor Management

vendors — Supplier master. company_name required. GST unique. Remarks field. Soft delete.

vendor_contacts — Multiple contact persons per vendor: name, phone, email, designation.

vendor_product_mapping — Links vendors to products they supply. Captures vendor-specific purchase_price and vendor_sku.

Product Management

product_categories — Self-referencing hierarchy via parent_id. Supports nested categories.

units_of_measure — Reference table for UOM codes (KG, PCS, L, BOX, etc.). Unique code.

products — Product catalog. SKU unique. Denormalized price fields for fast lookup. Reorder threshold. Remarks. Soft delete.

pricing — Versioned pricing: purchase_price, dealer_landing_price, selling_price with effective date range.

Inventory Management

stock_on_hand — Current physical quantity per product (no warehouse). Unique product_id. Updated on dispatch and stock-in.

stock_reserved — Quantity committed to confirmed orders. Unique product_id. Updated on order confirm/cancel/deliver.

stock_damaged — Damaged quantity per product. damage_reason, recorded_by (FK→users), remarks. Excluded from available stock.

stock_transactions — Immutable ledger of every stock movement: type, reference, quantity_change, quantity_after, unit_cost.

inventory_adjustments — Manual adjustment records with reason, approved_by, before/after quantities, and remarks.

Sales Management

orders — Order header. Captures cancelled_by, cancelled_at, cancellation_reason, credit_hold flag, remarks. Soft delete.

order_items — Line items. Snapshots: unit_price, dealer_landing_price, selling_price at order time. Remarks.

order_status_history — Append-only status audit: every status transition with actor and timestamp.

Challan Management

challans — Delivery note linked to order. Multiple challans per order for partial deliveries. Remarks.

challan_items — Items per challan linked to order_item. Quantity, UOM, remarks.

Dispatch Management

dispatches — Dispatch record linked to challan AND order. vehicle_number, driver_name, driver_phone inline. Remarks. No separate vehicle/driver tables in V1.

dispatch_items — Products dispatched per dispatch: product_id, quantity_shipped, UOM, remarks.

Returns Management

returns — Return header linked to original order and customer. Reason, status, remarks.
return_items — Items per return: product, quantity, condition (GOOD/DAMAGED/EXPIRED). Stock routing depends on condition. Remarks.

Finance

payments — Payment records: mode (CASH/UPI/RTGS/CHEQUE/CARD/WALLET), reference_number, status, received_by. Remarks.
outstanding_balances — Running summary per customer: total_invoiced, total_paid, balance. Unique customer_id.

Notifications

notifications — Notification events: type, severity, title, message, related entity, created_by.
notification_recipients — Per-user delivery state: is_read, read_at, delivered_at. 1 notification → N recipients.

Audit & Compliance

audit_logs — Append-only. Captures user_id, action, entity_type, entity_id, old_values (JSON), new_values (JSON), ip_address. Never updated or deleted.

Reorder Intelligence

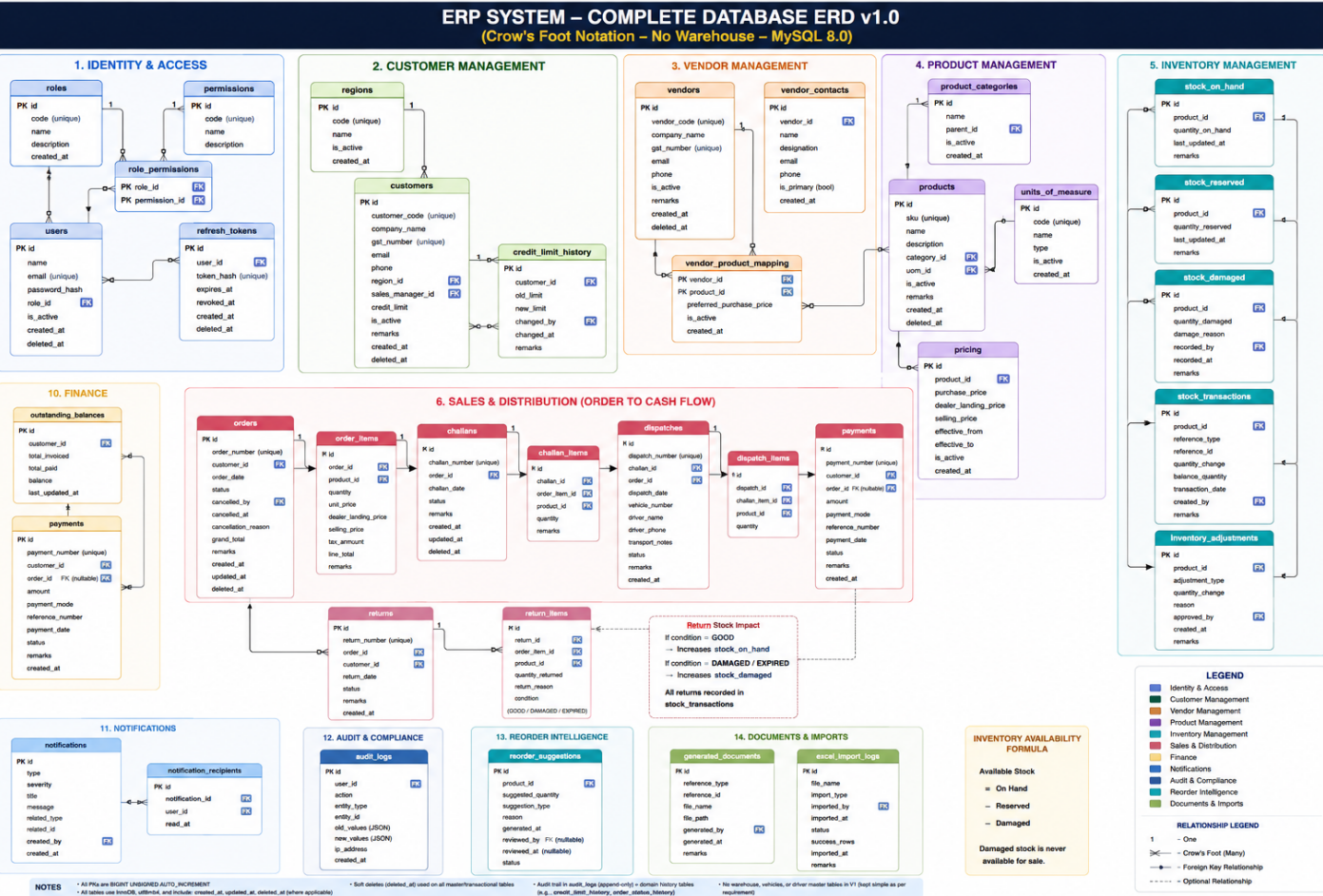
reorder_suggestions — Auto-generated reorder alerts per product. suggested_quantity, reason, type, status, reviewed_by.

Documents & Imports

generated_documents — Tracks generated PDF/Excel files: type, reference, file path, generated_by.
excel_import_logs — Tracks Excel import operations: import_type, file_name, total/success/error rows, uploaded_by.

5. Entity Relationship Diagram

The following ERD is the authoritative Version 1.0 schema diagram, reflecting all finalized design decisions: no warehouse tables, vehicle/driver details inline in dispatches, stock_damaged as a dedicated table, and remarks on all business tables. All modules are color-coded; Crow's Foot notation is used throughout.



6. Table-by-Table Design

Table: users

Core system user accounts for all actors: Sales Managers, Admins, Warehouse Staff.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK AUTO_INCREMENT
name	VARCHAR(150)	Full display name
email	VARCHAR(200)	UNIQUE — login identifier
password_hash	VARCHAR(255)	Bcrypt hash
role_id	BIGINT UNSIGNED	FK → roles.id
phone	VARCHAR(20)	Optional
is_active	TINYINT(1)	DEFAULT 1, CHECK IN (0,1)
last_login_at	DATETIME	NULL — updated on auth
deleted_at	DATETIME	Soft delete
created_at / updated_at	DATETIME	Timestamps

Indexes: UNIQUE(email); idx_users_role_id; idx_users_is_active

Design Notes: Soft delete. last_login_at updated on JWT auth. No hard deletes in production.

Table: customers

Master buyer registry. Supports auto-lookup and auto-creation workflow.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
code	VARCHAR(50)	UNIQUE — system-generated
name	VARCHAR(200)	Contact person name
company_name	VARCHAR(200)	Required — legal name
gst_number	VARCHAR(20)	UNIQUE — NULL allowed for unregistered
region_id	BIGINT UNSIGNED	FK → regions.id
phone / email	VARCHAR	Contact details
billing_address / shipping_address	TEXT	Address fields
credit_limit	DECIMAL(14,2)	DEFAULT 0.00
sales_manager_id	BIGINT UNSIGNED	FK → users.id
is_active	TINYINT(1)	DEFAULT 1
remarks	TEXT	NULL
deleted_at	DATETIME	Soft delete

Indexes: UNIQUE(code), UNIQUE(gst_number); idx_customers_region_id; idx_customers_sales_manager_id; idx_customers_name

Design Notes: Auto-lookup queries customers by name. If found: autofill. If not: create new with region and sales_manager assignment.

Table: regions

Geographic region reference for customer segmentation and regional reporting.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK

Column	Data Type	Notes
name	VARCHAR(100)	UNIQUE
code	VARCHAR(20)	UNIQUE — short code
description	TEXT	NULL
is_active	TINYINT(1)	DEFAULT 1
created_at / updated_at	DATETIME	Timestamps

Indexes: UNIQUE(name), UNIQUE(code)

Design Notes: Reference table. 1 region → N customers. No soft delete — system-level reference data.

Table: products

Product master catalog with denormalized pricing for fast order-time lookups.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
sku	VARCHAR(100)	UNIQUE
name	VARCHAR(300)	Display name
description	TEXT	NULL
category_id	BIGINT UNSIGNED	FK → product_categories.id
vendor_id	BIGINT UNSIGNED	FK → vendors.id (primary supplier)
unit_of_measure_id	BIGINT UNSIGNED	FK → units_of_measure.id
purchase_price	DECIMAL(12,2)	Denormalized current-state cache
dealer_landing_price	DECIMAL(12,2)	Denormalized current-state cache
selling_price	DECIMAL(12,2)	Denormalized current-state cache
reorder_threshold	INT UNSIGNED	Triggers reorder_suggestion
remarks	TEXT	NULL
is_active	TINYINT(1)	DEFAULT 1
deleted_at	DATETIME	Soft delete

Indexes: UNIQUE(sku); idx_products_category_id; idx_products_vendor_id

Design Notes: Versioned pricing history stored in pricing table. Fields here are denormalized for performance.

Table: stock_on_hand / stock_reserved / stock_damaged

Three snapshot tables maintaining current inventory state per product (no warehouse dimension in V1).

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
product_id	BIGINT UNSIGNED	FK → products, UNIQUE
quantity_on_hand / _reserved / _damaged	INT UNSIGNED	Current count
damage_reason	TEXT	Only on stock_damaged
recorded_by	BIGINT UNSIGNED	FK → users — only on stock_damaged
remarks	TEXT	Only on stock_damaged and stock_reserved
last_updated_at	DATETIME	Denormalized refresh timestamp

Indexes: UNIQUE(product_id) on all three tables

Design Notes: Available Stock = on_hand – reserved – damaged. Damaged stock never available for sale.

Table: stock_transactions

Immutable append-only ledger of every stock movement event.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
product_id	BIGINT UNSIGNED	FK → products
transaction_type	VARCHAR(40)	STOCK_IN / RESERVED / RELEASED / DISPATCHED / RETURNED / DAMAGED / ADJUSTED / RECONCILED
reference_type	VARCHAR(40)	ORDER / CHALLAN / DISPATCH / RETURN / ADJUSTMENT
reference_id	BIGINT UNSIGNED	Polymorphic FK
quantity_change	INT	Signed — positive or negative
quantity_after	INT UNSIGNED	Post-transaction balance
unit_cost	DECIMAL(12,2)	Cost at time of movement
notes	TEXT	NULL
created_at	DATETIME	Append-only — no updated_at

Indexes: idx_st_product_type; idx_st_reference; idx_st_created_at

Design Notes: NEVER UPDATE OR DELETE. Partition by RANGE(YEAR(created_at)) for performance at scale.

Table: orders

Order header — the central hub of the order-to-cash workflow.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
order_number	VARCHAR(50)	UNIQUE — system-generated
customer_id	BIGINT UNSIGNED	FK → customers
sales_manager_id	BIGINT UNSIGNED	FK → users
order_date	DATE	Business date
status	VARCHAR(30)	DRAFT/CONFIRMED/ALLOCATED/PARTIALLY_DISPATCHED/DISPATCHED/D ELIVERED/CANCELLED
payment_status	VARCHAR(20)	PENDING/PARTIAL/PAID
subtotal / discount_total / tax_total / grand_total	DECIMAL(14,2)	Order financials
credit_hold	TINYINT(1)	1 blocks dispatch
cancelled_by / cancelled_at / cancellation_reason	mixed	NULL unless cancelled
remarks	TEXT	NULL
deleted_at	DATETIME	Soft delete

Indexes: UNIQUE(order_number); idx_orders_customer_date; idx_orders_status; idx_orders_payment_status

Design Notes: Status transitions tracked in order_status_history. Cancellation only allowed in pre-dispatch states.

Table: order_items

Line items per order — captures price snapshot at time of order creation.

Column	Data Type	Notes
id	BIGINT UNSIGNED	PK
order_id	BIGINT UNSIGNED	FK → orders
product_id	BIGINT UNSIGNED	FK → products
quantity	INT UNSIGNED	Units ordered
unit_price	DECIMAL(12,2)	Snapshot — cost price
dealer_landing_price	DECIMAL(12,2)	Snapshot — dealer cost
selling_price	DECIMAL(12,2)	Snapshot — list price
discount	DECIMAL(12,2)	Line discount
tax_amount	DECIMAL(12,2)	GST / tax
line_total	DECIMAL(14,2)	Computed: (selling_price – discount) * qty + tax
remarks	TEXT	NULL

Indexes: idx_order_items_order_id; idx_order_items_product_id

Design Notes: Price columns are immutable snapshots. Master pricing changes do not retroactively alter order_items.

Dispatch Workflow

Dispatch records represent the physical movement of goods. In V1, vehicle and driver details are stored inline within the dispatches table. No separate vehicle or driver master tables exist.

- **Creation:** Dispatch created when challan is ready. Linked to challan_id and order_id.
- **Assignment:** Vehicle number, driver name, and driver phone entered inline.
- **Shipping Address:** Destination address captured on dispatch record for delivery proof.
- **Dispatch Items:** Products and quantities confirmed in dispatch_items (product_id, quantity_shipped, UOM, remarks).
- **Status Flow:** PENDING → ASSIGNED → IN_TRANSIT → DELIVERED or CANCELLED.
- **Stock Impact on Delivery:** On DELIVERED: stock_on_hand -= quantity; stock_reserved -= quantity; stock_transactions DISPATCHED entry.
- **V2 Plan:** introduce vehicles and drivers master tables with maintenance tracking, licensing, and route management.

12. Returns Workflow

Stage	Description
Initiation	Customer requests return. Return officer creates returns record: order_id, customer_id, return_date, reason, remarks.
Approval	Returns Manager reviews and updates status: APPROVED or REJECTED.
Item Entry	return_items added per product: quantity, condition (GOOD/DAMAGED/EXPIRED), notes, remarks.
GOOD Condition	Stock restocked: stock_on_hand += quantity. RETURNED ledger entry in stock_transactions.
DAMAGED Condition	Stock logged as damaged: stock_damaged += quantity. DAMAGED ledger entry in stock_transactions.
EXPIRED Condition	Stock not restocked. Disposal workflow initiated. ADJUSTED ledger entry with reason = EXPIRED_DISPOSAL.
Financial Credit	Approved returns may trigger credit note or refund in payments table (payment_mode = REFUND).
Status Flow	REQUESTED → APPROVED → RECEIVED → RESTOCKED / REJECTED.

13. Finance Workflow

Stage	Description
Invoice Creation	On order DELIVERED: outstanding_balances.total_invoiced += order.grand_total; balance recalculated.
Payment Recording	Payment received → payments record created: amount, mode, reference_number, received_by, status=COMPLETED.
Balance Update	outstanding_balances.total_paid += payment.amount; balance = total_invoiced – total_paid; last_updated_at refreshed.
Credit Limit Check	On order creation: if outstanding_balance > credit_limit → orders.credit_hold = 1. Alert notification generated.
Credit Limit Change	When limit changed: credit_limit_history record inserted (old_limit, new_limit, changed_by, reason). customers.credit_limit updated.
Advance Payments	payments.order_id = NULL for advance payments not linked to specific orders.
Payment Modes	CASH, UPI, RTGS, CHEQUE, CARD, WALLET — CHECK constraint enforced.
Refunds	Return-triggered refunds: payments record with negative amount or separate REFUNDED status.

14. Notifications & Audit Strategy

Notification Triggers

Trigger	Event
ORDER_CONFIRMED	Order status changed to CONFIRMED
PAYMENT_RECEIVED	Payment status = COMPLETED
CREDIT_LIMIT_EXCEEDED	Outstanding balance > credit_limit
STOCK_LOW	stock available < products.reorder_threshold
ORDER_CANCELLED	Order status = CANCELLED
DISPATCH_ASSIGNED	Dispatch created with vehicle and driver
RETURN_REQUESTED	Returns record created with status = REQUESTED
SYSTEM_ALERT	System-generated warnings and errors

Multi-Recipient: One notification event → N recipients via notification_recipients. Each recipient tracks read_at and delivered_at independently. Delivery channel (email/SMS/push) handled by an external service; this table tracks delivery confirmation.

Audit Strategy

- Trigger Points: CREATE, UPDATE, DELETE on customers, vendors, products, orders, payments, users, credit_limit_history, stock_damaged, inventory_adjustments.
- Captured Fields: user_id (actor), action, entity_type, entity_id, old_values (JSON), new_values (JSON), ip_address, created_at.
- Immutability: No UPDATE or DELETE on audit_logs — enforced via INSERT-only MySQL user role for audit service.
- Implementation: Application-layer audit service (not DB triggers) for testability and portability.
- Status Histories: order_status_history and credit_limit_history are domain-specific append-only audit tables in addition to audit_logs.
- Login Events: User login, logout, and failed attempts recorded as action = 'LOGIN' / 'LOGOUT' / 'FAILED_LOGIN'.
- Retention: Partition by YEAR(created_at). Partitions older than 7 years archived to cold storage.
- JSON Indexing: Generated columns on old_values/new_values for high-frequency attribute lookups.

15. Indexing & Scaling Strategy

Critical Indexes

Table	Index Name	Columns	Purpose
customers	idx_cust_name	name	Auto-lookup search
customers	idx_cust_region	region_id	Regional reports
orders	idx_ord_cust_date	customer_id, order_date	Customer order history
orders	idx_ord_status	status	Pipeline views
order_items	idx_oi_order	order_id	Items per order
order_items	idx_oi_product	product_id	Product sales reports
stock_transactions	idx_st_product	product_id, created_at	Stock movement history
stock_transactions	idx_st_type	transaction_type	Filtered ledger queries
payments	idx_pay_cust_date	customer_id, payment_date	Customer payment history
audit_logs	idx_audit_entity	entity_type, entity_id	Entity change lookup
audit_logs	idx_audit_user	user_id	Actor activity log
challans	idx_chal_order	order_id	Challans per order
dispatches	idx_disp_challan	challan_id	Dispatch per challan
returns	idx_ret_order	order_id	Returns per order

Scaling Strategy

- Read Replicas: MySQL read replicas serve all SELECT queries for reports, dashboards, and analytics. Master handles writes only.
- Connection Pooling: ProxySQL manages connection pooling. Target: ≤200 connections on primary.
- Redis Caching: available stock per product (TTL 30s), customer credit status (TTL 60s), product pricing (TTL 5m).
- Table Partitioning: RANGE partition by YEAR(created_at) on stock_transactions, audit_logs, orders, payments.
- Archival: Records >3 years moved to archive schema. Application queries include archive fallback for old data.
- Async Processing: Notification delivery, audit log writes, reorder computations run via Redis Queue / RabbitMQ asynchronously.
- Index Maintenance: ANALYZE TABLE weekly. Slow query log with 1s threshold. Monthly index review cycle.
- Future Sharding: If multi-tenant, add company_id to all tables. Shard by company_id range.

16. Future Enhancements

Enhancement	Description
Warehouse Module	Add warehouses table. Extend stock_on_hand, stock_reserved, stock_damaged with warehouse_id for multi-location inventory. Planned V2.
Vehicle & Driver Masters	Introduce vehicles and drivers master tables with maintenance records, licensing, and route management. V2 will FK dispatches.vehicle_id / driver_id.
Purchase Orders	Full procure-to-pay: purchase_orders, purchase_order_items, goods_receipt tables.
GST Compliance	Expand tax_amount to CGST, SGST, IGST split columns. Add hsn_code to products.
Multi-Currency	Add currency_code and exchange_rate_snapshot to orders and payments. Create exchange_rates reference table.
Barcode / QR	Add barcode column to products. Batch scanning for dispatch and return processing.
Driver Mobile App	Extend dispatches with GPS delivery_lat, delivery_lng, delivery_proof_image_url.
AI Reorder Optimization	Enhance reorder_suggestions with ML-predicted demand_forecast, lead_time_days, safety_stock.

Enhancement	Description
Customer Portal	Customer-facing order tracking via read-only API on orders, challans, dispatch tables.
Multi-Tenant SaaS	Add company_id tenant key to all tables. Row-level security via application middleware.

This document is the complete Version 1.0 Database Design Specification for the Wholesale & Distribution ERP System. The ERD and all written sections are fully synchronized and serve as the source of truth for all development, review, and architecture decisions.

Version 1.0 · June 2026 · Confidential · Database Architecture Team